Munshi — Complete Project Documentation

HackWithHyderabad 3.0 · AI agent with persistent memory (Hindsight) · Nov 6, 2026 · @Prajwal

1. Executive summary

Munshi is an AI collections agent for small Indian B2B suppliers. It learns why each customer pays late, catches those problems before an invoice is sent, and notices when what it learned stops being true.

It runs on Hindsight for memory, Band for multi-agent coordination and Groq-hosted models for reasoning.

One loop, three moments

1. **Prevent.** Before an invoice goes out, Munshi checks it against this buyer's past holds and rejections. It flags likely blockers and shows the evidence.
2. **Follow up.** When an invoice is overdue, Munshi drafts the right message to the right person at the right time. The owner approves every message.
3. **Revise.** When a contact, requirement or payment habit changes, Munshi flags the conflict instead of trusting old memory. The Risk agent warns the owner before a default.

Why memory is essential. What gets a supplier paid lives in conversations, not database fields: payment-run days, who actually releases funds, the document a buyer always asks for, the first signs of trouble. Hindsight turns those conversations and their outcomes into evolving beliefs with evidence. Without it, Munshi is a reminder bot.

Why it can win

- It uses Hindsight's learning layer: experience facts, observations with trends, mental models, directives and disposition. Most teams use only retain and recall.
- It proves learning with a three-way comparison (no memory, structured database memory, Hindsight) on held-out months, shown as a learning curve.
- It shows memory that unlearns: a belief weakens and the agent changes its behaviour.
- It uses Band as the product's collaboration layer, not a checkbox.
- It solves a national problem judges recognise at once: delayed payments to MSMEs.

Scope. One persona (the owner or accountant of a small manufacturer), one workflow (invoice to cash), simulated customers based on public data, and human approval on every outgoing message.

2. Problem, persona and market

Small Indian suppliers wait about 73 days to get paid. The knowledge that would speed this up sits in one person's head and in scattered WhatsApp chats.

Figure	Value	Source
Average national invoice cycle	73 days	Recordent SME Receivables Report 2026, via KNN
Invoices issued with 0 to 30 day credit	82.6%	Same report (about 1.1 lakh MSMEs)
Unpaid MSME invoices, as of March 2024	₹7.34 lakh crore (estimate)	Whalesbook
Legal limit for paying micro and small suppliers	45 days with a written agreement, 15 without	Whalesbook, TaxGuru

The Recordent report attributes delays to weak collections and payment discipline, not long credit terms. That is the gap Munshi targets.

Persona

Ravi owns a packaging unit near Hyderabad with about 80 business customers. Invoices are made in Tally, and payments are chased on WhatsApp and phone by Ravi and one accountant.

- He remembers which customer pays on which day and whom to call, but not reliably across 80 customers.
- Invoices get held for a missing PO number, an unsigned challan or a wrong GSTIN. He finds out weeks later.
- Contacts change without notice, so reminders go to the wrong person.
- Early signs of a customer in trouble (excuses, part payments, slipping dates) are noticed too late.
- When the accountant leaves, the knowledge leaves too.

His job to be done: get paid sooner without damaging relationships, know why each invoice is stuck, and see trouble early.

Positioning

Category	Examples	What they do	Why Ravi doesn't use it
Enterprise AR platforms	HighRadius, Upflow, Billtrust	Payment prediction, prioritisation, dunning. Upflow can generate agent instructions from six months of conversations; HighRadius flags deviations from expected payment behaviour.	Built and priced for finance teams running ERPs
Small-business accounting and reminder tools	Tally add-ons, bookkeeping apps	Scheduled payment reminders	Rule-based; they don't learn from replies or outcomes
Munshi	This project	Learns a playbook per customer from conversations and outcomes, catches blockers before sending, revises its beliefs, owner approves every message	Built for WhatsApp-first MSMEs

Honest claim. Munshi is not the first AI collections tool. Its claim is narrower: continuous learning from outcomes, explicit belief revision and pre-send blocker memory, for suppliers who will never buy enterprise AR software.

3. The solution: Prevent, Follow up, Revise

Every invoice passes through the same loop. Each pass leaves Munshi knowing a little more about that customer, and nothing reaches a customer without the owner's approval.

Moment	Trigger	Fixed rules check	Memory provides	Output	Retained afterwards
Prevent	Invoice created or imported	GSTIN format, checksum and buyer match; PO present; amount within PO; duplicate number	Documents and details this buyer has required, past hold reasons, recent changes	Readiness result (Ready, Blocker, Conflict, Uncertain) with evidence	The owner's decision and any fix
Follow up	Daily check finds an invoice near due or overdue	Allowed hours, reminder frequency cap	Right contact, payment-run days, what worked or failed, promises made	Draft message with recipient, timing and attachments, waiting for approval	The sent action, the customer's reply, the payment
Revise	Every new reply, payment or day	Days late compared with history	Observation trends, conflicting facts, distress language	Risk level change, conflict flag, alert to the owner	The owner's confirmation of the new belief

Readiness results

- **Ready:** all rules pass and no learned requirement is missing.
- **Blocker:** a rule fails, or a learned requirement backed by evidence is missing, such as a signed challan.
- **Conflict:** memory disagrees with the buyer record or with newer evidence. Example: a recent message says the PO requirement was dropped.
- **Uncertain:** the evidence is thin: a single mention, an old fact, or observations still being consolidated.

Risk levels

- **Normal:** payment beliefs stable or strengthening.
- **Watch:** one payment belief weakening, or one distress cue in a reply.
- **High:** two or more signals together, such as missed payment runs plus distress language or part payments. Munshi suggests holding the next dispatch, asking for an advance or shortening credit. The owner decides.

Worked example (illustrative)

1. **Month 1.** An invoice to Reddy Traders goes out without a PO number and is held. Priya from their accounts team replies that every invoice needs the PO number.

2. **Month 2.** Prevent flags the missing PO before sending. Munshi reminds Priya on the 3rd, and the invoice is paid on the 5th, their payment-run day.
3. **Month 3.** The belief "pays on the 5th if reminded before it" strengthens with each payment.
4. **Month 4.** Priya says Rahul now handles payables, and a reply mentions tight cash flow. The 5th and 20th pass without payment. The belief weakens, the risk level goes to High, and the Risk agent alerts the owner in Band.

For real-world use, an approved message opens as a pre-filled WhatsApp link that the owner sends from their own phone. There is no automatic sending.

4. Hackathon alignment

Munshi is designed against the judging weights: innovation and memory carry 55 of the 100 points, so memory does the core work, not a side feature.

Criterion	Weight	How Munshi earns it	What judges see
Innovation	30	Memory that learns from outcomes and revises its beliefs; prevention before chasing; multi-agent desk for MSMEs	A belief weakening and the agent changing behaviour
Hindsight memory	25	Experience facts, observations with trends, mental models, directives, disposition, evidence on every suggestion	"Why this?" panel; the three-version learning curve
Technical implementation	20	Rules engine, Hindsight integration, Band agents, a reproducible simulator and evaluation harness	Clean repo, tests, one-command evaluation
User experience	15	Customer playbook cards, belief timeline, one-click approvals, WhatsApp hand-off	60-second demo with no explaining needed
Real-world impact	10	Delayed MSME payments: a 73-day average cycle	Days late and early-warning lead time, measured

Mandatory and suggested technology

Requirement from the PDF	How it is met
Hindsight (mandatory)	Core memory engine; Cloud for the demo, local Docker for development
Band AI (download and use)	Collector and Risk agents collaborate with the owner in a Band room; the team also builds with Band Desktop
Any LLM; GPT-OSS-120B and Qwen3-32B suggested via Groq	gpt-oss-120b for agents, qwen3-32b for simulated customers
Realistic data	Simulated customers based on a public receivables dataset and supplier interviews
Learning curve across interactions	Months 1 to 3 learning, months 4 to 6 test, three versions compared
One workflow, one persona	Invoice to cash for a small manufacturer's owner
Not student-centric	B2B finance operations

Submission deliverables

A clean, documented GitHub repo, a demo video, a live demo, an article, social post and video from every team member, and a clear explanation of how Hindsight is used. See section 22 for the checklist.

5. Product walkthrough and screens

The app has seven screens. The first five are the product; the last two exist for the demo and the evaluation.

Screen	Purpose	Key elements
Dashboard	See where money is stuck	Total receivable, overdue amount, customers at Watch or High risk, drafts waiting for approval, blockers caught this month

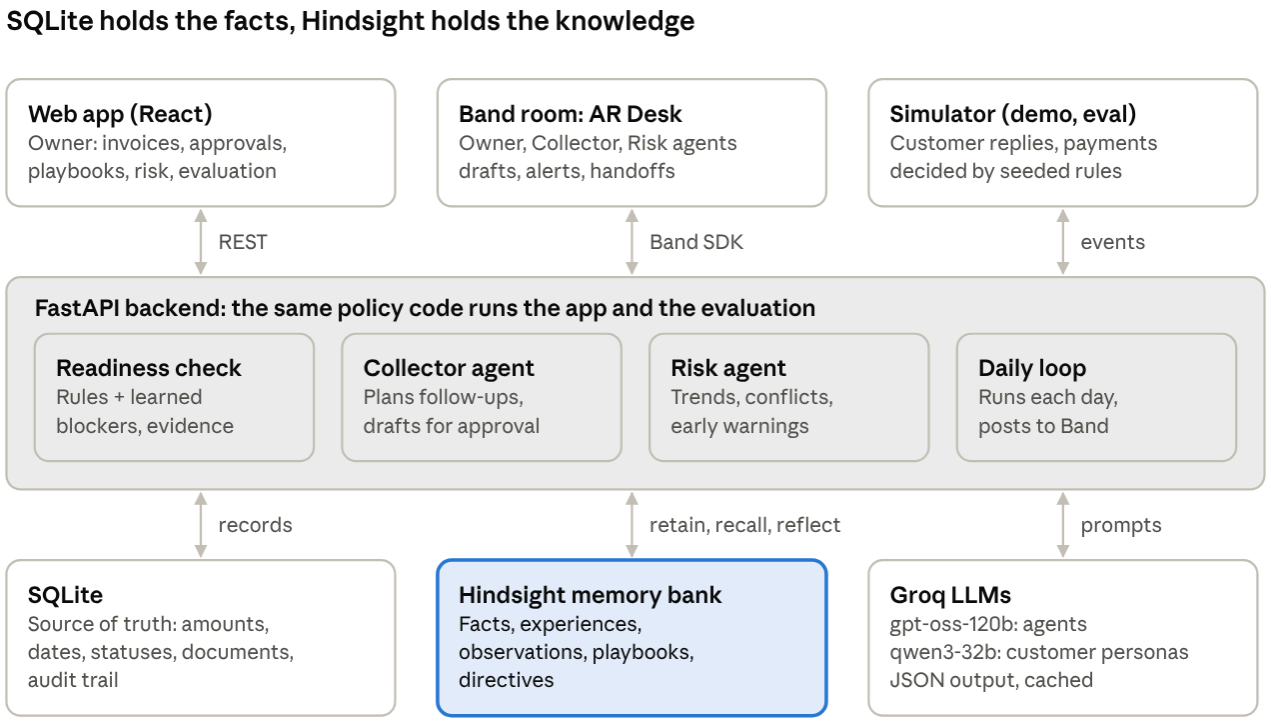
Screen	Purpose	Key elements
Customer page	Everything Munshi knows about one customer	Playbook card (the mental model), beliefs with trend arrows, timeline of invoices, messages, payments, contacts
New invoice and readiness check	Catch blockers before sending	Invoice form, document uploads tagged by type, readiness panel separating rule findings from memory findings, evidence pop-overs
Approvals inbox	Owner approves every outgoing message	Draft text, recipient, timing, attachments, a "Why this?" panel, approve, edit or reject, an "Open in WhatsApp" link
Risk alerts	Early warning	Risk level changes with the signals behind them and the suggested action
Simulation console	Run the demo	Simulated clock with fast-forward, inject events such as a contact change, chat as the customer
Evaluation	Show the proof	Learning curves for versions A, B and C, early-warning lead time, blocker catch rate

Main user journey

1. Ravi imports customers and open invoices from a Tally-style CSV export.
2. He creates a new invoice. The readiness panel shows a blocker: "Last two invoices to Reddy Traders were held until a signed challan arrived." He attaches the challan.
3. When the invoice is due, a draft reminder appears in the Approvals inbox and in the Band room. He opens "Why this?", sees the evidence, and approves.
4. The customer's reply and the payment are recorded, by the simulator in the demo or by Ravi pasting a WhatsApp reply in real use.
5. Over the following weeks the customer's playbook card updates. When a belief weakens, a risk alert appears and the Risk agent posts in Band.

6. System architecture

One FastAPI backend sits between three surfaces (web app, Band room, simulator) and three services (SQLite, Hindsight, Groq). The agents are backend modules, not separate services.



Munshi architecture · 3 surfaces, 1 backend, 3 stores and services

The owner works in the web app or the Band room; both call the same backend. The backend writes every amount and status to SQLite, and every business event to Hindsight.

Source-of-truth rules

- **SQLite decides:** amounts, dates, invoice status, which documents are attached, what was approved and sent.
- **Hindsight informs:** why customers pay or don't, who to contact, what was required, what changed, how risky a customer looks.
- **Code computes:** rule checks, days late, missed runs, risk level from signals, business-hours limits.
- **The LLM writes:** message wording, JSON plans and summaries, always from evidence passed to it.

Data flow for one follow-up

1. The daily loop finds invoice INV-1042 two days before Reddy Traders' usual payment run.
2. The Collector calls reflect on the customer's memory and gets a plan with evidence ids.
3. Code clamps the time to business hours and checks the reminder cap; the LLM writes the message.
4. The draft is saved in SQLite and posted to the Band room and the web inbox.
5. The owner approves. The message is delivered, and the action is retained in Hindsight.
6. The customer's reply and the payment come back. Both are stored in SQLite and retained in Hindsight, linked to the reminder.
7. Hindsight consolidates the new facts; the playbook and beliefs update before the next decision.

7. Hindsight memory design

Hindsight holds what Munshi knows about customers; SQLite holds amounts, dates and statuses. Munshi never asks Hindsight for a number it must act on.

Hindsight concepts used here, from its docs: experience facts (the bank's own actions), observations consolidated from facts, mental models checked first during reflect, directives as hard rules, disposition traits, and `based_on` evidence on every reflect (memory types, learning capabilities, reflect). Pin one Hindsight version on Day 1 and check every name below against it.

Banks

Bank	Naming	Purpose
Production / demo	`munshi-<business-slug>`	One bank per supplier business; customers separated by tags and entities
Evaluation	`eval-<run-id>-C-s<seed>`	A fresh bank per evaluation run and seed, so runs never contaminate each other

Bank configuration

Setting	Value
Retain mission	Extract facts that affect when and how this supplier gets paid: each customer's payment-run days, who approves and releases payments, required invoice details and documents, reasons invoices were held or disputed, promises to pay and whether they were kept, contact changes, and signs of financial stress. Ignore greetings and small talk.
Observations mission	Consolidate durable patterns per customer: payment timing, which follow-ups work or fail, document requirements, and changes in behaviour. Note when newer evidence contradicts an older pattern.
Reflect mission	You are Munshi, the accounts clerk for {business}. Help the owner get paid on time without harming customer relationships. Base every recommendation on evidence and say when evidence is thin or stale.
Disposition	Skepticism 4, literalism 3, empathy 3. Promises to pay are cheap, so the bank should be sceptical.

Directives (global, untagged)

1. Never threaten legal action or use coercive language. Any escalation beyond a reminder needs owner approval.
2. Never recommend contacting a customer outside 09:00 to 19:00 IST.
3. Never reveal one customer's information to another.
4. Never state that an invoice is valid or that a payment was received; those come from the ledger.
5. Cite the evidence behind every recommendation. If there is none, say so.
6. Mention MSME payment-term rules only when the owner has marked the business as a registered micro or small enterprise.

What gets retained

One business event per retain call. Each call sets the event date as the timestamp, the event id as the document id (so a retry overwrites instead of duplicating), and tags `customer:<id>` and `kind:<event>`.

Event	Example content	Expected fact type
Invoice sent	On 2026-03-01 I (Munshi) sent invoice INV-1042 for ₹48,200 to Priya at Reddy Traders, with PO 4471 and the signed challan attached.	Experience
Invoice held	On 2026-02-12 Reddy Traders held INV-1037 because the PO number was missing.	World
Reminder sent	On 2026-03-03 I reminded Priya on WhatsApp about INV-1042, two days before their usual payment run.	Experience
Customer reply	On 2026-03-03 Priya (accounts, Reddy Traders) replied: "Payment run is on the 5th, will release with this batch."	World
Promise to pay	On 2026-04-21 Rahul (Reddy Traders) promised payment of INV-1060 by 2026-04-28.	World
Payment received	On 2026-03-05 Reddy Traders paid INV-1042 in full, 4 days late and 2 days after my reminder to Priya.	World, linked to the reminder
Contact change	On 2026-04-02 Priya said Rahul now handles payables at Reddy Traders.	World
Owner confirmation	Owner confirmed on 2026-04-03: Rahul is the payables contact at Reddy Traders.	World, high authority

Writing rules:

- Phrase Munshi's own actions in the first person ("I sent", "I reminded") so they are extracted as experience facts.
- Always use absolute dates and full names with roles. Never write "yesterday" or "he".
- Quote customer words briefly; don't paste whole chat logs.
- Never retain bank account numbers or phone numbers; those stay in SQLite.
- Never retain a draft that wasn't sent.

How each part of Munshi uses memory

Caller	Operation	Query (shortened)	Result used for
Readiness check	Reflect, tagged to the customer	What details or documents has this customer required, why were past invoices held, and has any requirement changed? Return JSON.	Memory findings in the readiness result
Collector agent	Reflect	Invoice X is N days overdue. Best next step: whom to contact, when, with what attachments and tone? Cite evidence.	The draft plan
Collector agent	Recall, experience facts only	Previous reminders to this customer and their outcomes	Frequency cap and history
Risk agent	Recall, observations	This customer's payment behaviour	Trend check
Risk agent	Reflect	Has payment behaviour, contact or requirements changed recently? Rate Normal, Watch or High with evidence. Return JSON.	Risk level
Customer page	Mental model	(stored query, below)	Playbook card
Approvals inbox	Reflect `based_on`	(from the Collector's reflect)	"Why this?" panel

Mental models

One mental model per customer, named "Playbook: {customer}", set to refresh when observations change (learning capabilities). Its source query: *How do we get paid on time by {customer}? Cover who to contact, payment-run days, required invoice details and documents, what follow-up has worked or failed, open promises, and current risk, with evidence.* This becomes the playbook card in the UI.

Trends and staleness

Hindsight's docs describe a computed trend on each observation (stable, strengthening, weakening, stale) based on when its evidence arrived (docs). The Risk agent reads that trend where the installed version exposes it. If not, it asks reflect to compare evidence before and after a date.

Observations and mental models are updated in the background and can lag new writes (freshness). The UI shows "memory updating" while consolidation is pending. The evaluation waits for consolidation before each simulated day's decisions.

Must verify in the Day 1 to 2 test

1. Does retain use the supplied event timestamp for temporal recall and trends? Simulated months depend on it. Fallback: put the date in the text and have the Risk agent compare dates itself.
2. Are first-person actions classified as experience facts? Fallback: tag them `kind:action` and filter by tag.
3. Do observations form per customer and change after contradicting evidence? Fallback: reflect-generated beliefs retained back as owner-style summaries.
4. Do mental models refresh on their own? Fallback: refresh them after each simulated week.
5. How long do retain and reflect take, and how many LLM calls do they cost? This sets the size of the evaluation.

8. Agents and Band integration

Two agents and the owner share one Band room. Agent logic lives in the backend, Band is the collaboration surface, and the evaluation runs the same logic without Band.

Band is a platform where AI agents and humans collaborate in shared rooms, with a Python SDK for connecting your own agents (band-sdk on PyPI, connect an agent). Its desktop app launched on 15 October 2026 (announcement).

Agents

Agent	Triggered by	Reads	Does	Never does
Collector	Daily check; an invoice reaching due or overdue; a new reply	Reflect and recall on the customer; ledger state	Plans and drafts follow-ups and posts them for approval; asks the Risk agent when a reply contains distress cues	Sends anything without approval
Risk	Each new payment, reply or day; a request from the Collector	Observations and trends; reflect on change; ledger lateness	Sets the risk level, flags conflicts, alerts the owner with evidence and a suggested action	Changes credit terms or holds dispatch itself
Readiness check (a service, not an agent)	Invoice created or edited	Rules plus reflect on the customer	Returns Ready, Blocker, Conflict or Uncertain	Approves an invoice on memory alone

A Dispute agent (GST and quantity disputes) is a stretch goal only.

Room design

Room: **AR Desk: {business}**. Participants: the owner (human), Collector, Risk.

Message conventions, so commands are parsed by rules, not an LLM:

- Collector posts drafts as: `[DRAFT A-231] To Priya (Reddy Traders), WhatsApp, today 10:30. <message>. Why: <one line of evidence>. Reply: approve A-231 / edit A-231: <text> / reject A-231`.
- Risk posts alerts as: `[RISK] Reddy Traders: Watch → High. Signals: missed 5th and 20th payment runs; "cash flow tight" on 21 Apr. Suggest: hold next dispatch or ask 30% advance. @owner`.
- Collector hands off to Risk with an @mention: `@Risk reply from Rahul mentions cash-flow trouble on INV-1060, please assess`. Risk answers in the same room.
- Owner questions in plain language ("why is Reddy at High?") go to the backend's explain endpoint, which calls reflect and replies with evidence.

Approval flow

1. The Collector creates a draft in SQLite with status DRAFTED.
2. The draft appears in both the Band room and the web Approvals inbox.
3. The owner approves, edits or rejects from either place. Both call the same backend endpoint; the first decision wins and the other surface updates.
4. On approval, the message is delivered: to the simulated customer in the demo, or as an "Open in WhatsApp" link in real use.
5. Only then is the action retained in Hindsight as an experience fact.

Implementation pattern

- `band/run_agents.py` starts both agents with the Band SDK as external agents (Python 3.11+, `band-sdk`).
- Inbound room messages are matched against the command patterns above. Matches call backend endpoints; anything else goes to the explain endpoint.
- Outbound posts come from backend events (new draft, risk change) through a small bridge that posts to the room.
- Setup: create a Band account, create two External Agents (Collector, Risk), copy each agent's id and API key into `.env`, create the room and add the owner.
- Confirm the exact SDK class names and adapter options against the Band docs on Day 1; the code in section 14 marks where.

Fallback and development use

If Band is unavailable, the web Approvals inbox and Risk page do the same job. Keep a screen recording of the Band flow for the demo. During development, run the team's coding agents in Band Desktop and capture screenshots for the articles.

9. Blocker engine: invoice readiness checks

Structured fields are checked by fixed rules; memory only adds which extra requirements this buyer has and why. An LLM never decides whether an amount, GSTIN or PO number is correct.

Rule checks

Code	Check	Result if it fails
R1_GSTIN_FORMAT	Buyer GSTIN matches the 15-character pattern	Blocker

Code	Check	Result if it fails
R2_GSTIN_CHECKSUM	The 15th character equals the computed check digit	Blocker
R3_GSTIN_MATCH	Invoice GSTIN equals the GSTIN in the buyer record	Blocker
R4_STATE_CODE	The first two digits match the buyer's state (sets IGST vs CGST + SGST)	Blocker
R5_PO_PRESENT	PO number present when the buyer record says PO is required	Blocker
R6_PO_AMOUNT	Invoice amount not above the PO amount (1% tolerance)	Blocker
R7_DUPLICATE	Invoice number not already used for this buyer	Blocker
R8_DUE_DATE	Due date equals invoice date plus the buyer's credit days	Warning
R9_REQUIRED_DOCS	Every document type in the buyer's confirmed list is attached	Blocker

GSTIN validation

The pattern is two state digits, a 10-character PAN, an entity number, the letter Z and a check character. The check character is a mod-36 checksum over the first 14 characters; the function below was tested against two published sample GSTINs.

```
import re

GSTIN_RE = re.compile(r"^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$")
CHARS = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def gstin_check_char(first14: str) -> str:
    total = 0
    for i, ch in enumerate(first14):
        product = CHARS.index(ch) * (1 if i % 2 == 0 else 2)
        total += product // 36 + product % 36
    return CHARS[(36 - total % 36) % 36]

def validate_gstin(gstin: str) -> list[str]:
    gstin = gstin.strip().upper()
    if not GSTIN_RE.match(gstin):
        return ["R1_GSTIN_FORMAT"]
    if gstin_check_char(gstin[:14]) != gstin[14]:
        return ["R2_GSTIN_CHECKSUM"]
    return []
```

Memory findings

The readiness service asks reflect, tagged to the customer, for requirements as JSON:

```
{
  "requirements": [
    {"code": "SIGNED_CHALLAN", "status": "required", "since": "2026-02-12",
     "evidence": ["<memory id>", "<memory id>"], "changed_recently": false}
  ],
  "conflicts": [
    {"about": "PO_REQUIRED", "old": "required", "new": "not required for
service invoices",
     "evidence": ["<memory id>"]}
  ]
}
```

Requirement codes are a fixed list: PO_REF, SIGNED_CHALLAN, GRN_COPY, EWAY_BILL, EINVOICE_IRN, CONTACT_PERSON, OTHER. The service then compares them with the invoice's attached document types using code, not the LLM.

Combining into one result

1. Any rule Blocker → **Blocker**.
2. Else any memory requirement marked required, with at least one evidence id and not attached → **Blocker** ("learned" badge).

3. Else any conflict between memory and the buyer record, or between old and new memory → **Conflict**; the owner confirms, and the confirmation is retained.
4. Else any requirement with a single piece of evidence, or evidence older than 180 days, or pending consolidation → **Uncertain**.
5. Otherwise → **Ready**.

When the owner confirms a learned requirement, it is also written to the buyer record. From then on it is a rule (R5 or R9), and memory keeps watching for changes to it.

10. Data model

Twelve SQLite tables, defined with SQLModel. These are the source of truth for every amount, date and status; Hindsight is never asked for them.

Table	Key fields	Notes
business	id, name, gstin, state_code, msme_registered (bool), bank_id	One row per supplier; `bank_id` names its Hindsight bank
customer	id, business_id, name, gstin, state_code, credit_days, po_required (bool), required_docs (JSON list), contact_name, contact_role, contact_phone, risk_level, risk_updated_at	`po_required` and `required_docs` change only when the owner confirms
invoice	id, customer_id, number, issue_date, due_date, amount_paise, po_number, status, readiness, created_at	Amounts stored in paise (integers)
invoice_document	id, invoice_id, doc_type, file_path, uploaded_at	doc_type: INVOICE_PDF, PO_COPY, SIGNED_CHALLAN, GRN, EWAY_BILL, OTHER
readiness_check	id, invoice_id, run_at, result, findings (JSON)	Each finding: code, source (RULE or MEMORY), message, evidence ids
action	id, invoice_id, customer_id, kind, channel, recipient, body, planned_for, status, decided_by, decided_at, sent_at, memory_event_id	The Collector's drafts and sent messages

Table	Key fields	Notes
inbound_message	id, customer_id, invoice_id, channel, sender_name, body, received_at, memory_event_id	Customer replies, from the simulator or pasted by the owner
payment	id, invoice_id, amount_paise, paid_on, reference, memory_event_id	Part payments allowed
promise	id, invoice_id, promised_by, promised_date, kept (bool or null)	Filled from replies; checked by the Risk agent
risk_alert	id, customer_id, from_level, to_level, signals (JSON), suggestion, evidence (JSON), status, created_at	status: OPEN, ACKNOWLEDGED, DISMISSED
memory_event	id (evt_…), kind, customer_id, invoice_id, occurred_on, content_hash, retained_at, hindsight_document_id	Audit link between each record and what was retained
sim_run	id, arm, seed, config (JSON), metrics (JSON), started_at, finished_at	Evaluation runs only

Status values

- **invoice.status:** DRAFT, READY, SENT, HELD, PARTLY_PAID, PAID, DISPUTED, WRITTEN_OFF.
- **invoice.readiness:** READY, BLOCKER, CONFLICT, UNCERTAIN.
- **action.kind:** REMINDER, DOC_SEND, THANK_YOU, ESCALATION_REQUEST.
- **action.status:** DRAFTED, APPROVED, EDITED, REJECTED, SENT.
- **customer.risk_level:** NORMAL, WATCH, HIGH.

Dates in the simulator use a simulated clock (`services/clock.py`), so every `*_on` and `*_at` field holds simulated time in demo and evaluation runs.

11. API design

A FastAPI backend under `/api` , JSON in and out, no authentication in the prototype beyond a single demo business. The frontend polls every 3 seconds; no websockets.

Method	Path	Purpose
POST	/api/import/tally-csv	Import customers and open invoices from a Tally-style CSV
GET	/api/dashboard	Totals, overdue, at-risk customers, pending drafts, blockers caught
GET	/api/customers	List customers with risk level
GET	/api/customers/{id}	Customer record, playbook (mental model text), beliefs with trends, timeline
PATCH	/api/customers/{id}	Owner edits: contact, po_required, required_docs
POST	/api/invoices	Create an invoice; runs the readiness check
POST	/api/invoices/{id}/documents	Upload a document with its doc_type
POST	/api/invoices/{id}/check	Re-run the readiness check
POST	/api/invoices/{id}/confirm	Owner resolves a Conflict or Uncertain finding; retained as an owner confirmation
POST	/api/invoices/{id}/send	Mark sent (after readiness); retains an "invoice sent" event
GET	/api/actions?status=DRAFTED	Approvals inbox
POST	/api/actions/{id}/approve	Approve, optionally with edited text
POST	/api/actions/{id}/reject	Reject with an optional reason (retained as an owner note)
GET	/api/actions/{id}/explain	Evidence behind a draft (reflect `based_on`)
POST	/api/messages/inbound	Record a customer reply (simulator or pasted WhatsApp text)
POST	/api/payments	Record a payment
GET	/api/risk/alerts	Open risk alerts
POST	/api/risk/alerts/{id}/ack	Acknowledge or dismiss an alert
POST	/api/explain	Plain-language question about a customer; answered by reflect with evidence
POST	/api/sim/reset	Load a scenario and reset the clock

Method	Path	Purpose
POST	/api/sim/tick?days=N	Advance the simulated clock N days and run the daily loop
POST	/api/sim/inject	Inject an event: contact change, requirement change, distress start
GET	/api/eval/results	Latest evaluation results for the charts

Example: approve a draft

```
POST /api/actions/231/approve
{"edited_body": null, "decided_by": "owner"}

200 OK
{"id": 231, "status": "SENT", "sent_at": "2026-03-03T10:30:00+05:30",
 "whatsapp_link": "https://wa.me/91XXXXXXXXXX?text=...", "memory_event_id":
"evt_0193"}
```

Every state-changing endpoint writes to SQLite first, then retains the event in Hindsight. If the retain call fails, the event is queued in `memory_event` with no `retained_at` and retried by a background task.

12. Frontend

React 18 with Vite, TypeScript, Tailwind, TanStack Query for data fetching and Recharts for the evaluation charts. Seven pages and nine shared components.

Pages

Page	Route	Main components
Dashboard	/	StatTiles, AtRiskList, PendingApprovals
Customer	/customers/:id	PlaybookCard, BeliefList, Timeline
New invoice	/invoices/new	InvoiceForm, DocumentUploader, ReadinessPanel
Approvals	/approvals	ApprovalCard, EvidencePopover
Risk	/risk	RiskAlertCard
Simulation console	/sim	SimClock, EventInjector, CustomerChat
Evaluation	/eval	LearningCurve, LeadTimeBars

Key components

- **PlaybookCard:** the customer's mental model, rendered as short sections (contact, payment days, required documents, what works, risk). Shows "memory updating" when consolidation is pending.
- **BeliefList:** observations for the customer, each with a trend arrow (strengthening, stable, weakening, stale) and an evidence count. Clicking a belief opens its evidence.
- **Timeline:** invoices, Munshi's actions, customer replies and payments in date order, with icons per kind.
- **ReadinessPanel:** two columns, "Rules" and "Learned from history". Each finding has a badge (Blocker, Conflict, Uncertain) and evidence. Conflicts have Confirm buttons.
- **ApprovalCard:** draft text (editable), recipient, planned time, attachments, approve and reject buttons, and an "Open in WhatsApp" link after approval.
- **EvidencePopover:** the memories behind a suggestion, each with its date and the event it came from.
- **CustomerChat:** a WhatsApp-style thread where the presenter or a judge can type as the customer.
- **LearningCurve:** average days late per month for versions A, B and C on one chart, with the test months shaded.
- **LeadTimeBars:** days of early warning before a missed payment run, per version.

Design notes

- Plain, dense, readable: an accountant's tool, not a landing page. One accent colour for "learned from memory" so judges can see what Hindsight contributed on every screen.
- Indian formats throughout: ₹ with lakh grouping (₹1,25,000), DD MMM YYYY dates, IST times.
- Works at laptop width for the demo; mobile layout is out of scope.

13. File and folder structure

One repository with a Python backend (which also holds the simulator and evaluation) and a React frontend. Each top-level folder has one owner from section 18.

```
munshi/
├── README.md                      # problem, demo GIF, architecture,
Hindsight section, how to run, results
├── docker-compose.yml             # hindsight, backend, frontend
├── .env.example                   # every variable from section 19, no
secrets
├── Makefile                       # make dev, make seed, make eval, make
agents, make test
├── LICENSE
├── docs/
│   ├── architecture.md
│   ├── hindsight.md               # bank config, retain templates, queries,
what we verified
│   ├── evaluation.md              # protocol and results table
│   ├── demo-script.md
│   └── images/                    # screenshots, architecture diagram,
charts
├── backend/
│   ├── pyproject.toml             # uv project, Python 3.11+
│   ├── munshi/
│   │   ├── main.py                # FastAPI app, routers, startup (create
tables, ensure bank)
│   │   ├── config.py              # pydantic-settings, reads .env
│   │   ├── db/
│   │   │   ├── models.py          # SQLModel tables from section 10
│   │   │   ├── session.py
│   │   │   └── seed.py            # demo business and customers
│   │   ├── api/
│   │   │   ├── customers.py  invoices.py  actions.py  messages.py
│   │   │   ├── payments.py   risk.py      explain.py  imports.py
│   │   │   └── sim.py        eval.py      dashboard.py
```

```
|   |   ├── memory/
|   |   |   ├── client.py          # the ONLY module that imports
hindsight_client
|   |   |   ├── bank_setup.py      # missions, disposition, directives,
mental models
|   |   |   ├── events.py          # retain templates, one function per event
kind
|   |   |   ├── queries.py         # reflect and recall prompts
|   |   |   ├── parsing.py         # JSON extraction and validation of
reflect output
|   |   |   └── outbox.py          # retry queue for failed retains
|   |   ├── checks/
|   |   |   ├── gstin.py           # section 9 code
|   |   |   ├── rules.py           # R1 to R9
|   |   |   └── readiness.py       # combines rules and memory findings
|   |   ├── agents/
|   |   |   ├── collector.py       # decide, plan, draft
|   |   |   ├── risk.py            # assess, alert
|   |   |   ├── drafting.py        # message wording with directives
|   |   |   ├── policy.py          # Policy interface used by app and
evaluation
|   |   |   └── prompts/           # .md prompt files, versioned
|   |   ├── llm/
|   |   |   ├── groq_client.py
|   |   |   ├── cache.py           # disk cache keyed by prompt hash
|   |   |   └── json_utils.py
|   |   ├── band/
|   |   |   ├── run_agents.py      # starts Collector and Risk on Band
|   |   |   ├── bridge.py          # backend events -> room posts
|   |   |   └── commands.py        # approve / edit / reject parsing
|   |   └── services/
|   |       ├── clock.py           # real or simulated time
|   |       ├── daily_loop.py      # the per-day run: checks, drafts, risk
|   |       └── whatsapp_link.py
|   ├── sim/
|   |   ├── profiles.py            # hidden customer traits and generator
|   |   ├── calibrate.py           # public AR dataset -> lateness
distributions
|   |   ├── world.py               # deterministic outcome rules
|   |   ├── personas.py            # LLM or template replies
|   |   └── scenarios/
|   |       ├── demo.yaml          # 4 customers, scripted story
|   |       └── eval.yaml          # 12 customers, 6 months, regime changes
|   ├── eval/
|   |   ├── run_eval.py            # CLI: --arms A,B,C --seeds 1,2,3 --
replies template
```

```
|   |   ├── arms/
|   |   |   ├── arm_a_fixed.py
|   |   |   ├── arm_b_structured.py
|   |   |   └── arm_c_hindsight.py
|   |   ├── metrics.py
|   |   ├── report.py              # results.json and charts
|   |   └── results/               # committed results of the final run
|   ├── scripts/
|   |   ├── prewarm_demo_bank.py   # runs the demo scenario into a fresh bank
|   |   └── spike_one_customer.py  # the Day 1-2 go/no-go test
|   └── tests/
|       ├── test_gstin.py  test_rules.py  test_readiness.py
|       ├── test_world_rules.py    # simulator outcomes are deterministic
|       ├── test_events.py         # retain templates
|       └── test_memory_integration.py  # needs a running Hindsight
└── frontend/
    ├── package.json  vite.config.ts  tailwind.config.ts  tsconfig.json
index.html
    └── src/
        ├── main.tsx  App.tsx  types.ts
        ├── api/client.ts          # typed fetch wrappers
        ├── hooks/                 # useCustomer, useApprovals, useSimClock
        ├── pages/                 # Dashboard, Customer, InvoiceNew,
Approvals, Risk, SimConsole, Evaluation
        ├── components/            # PlaybookCard, BeliefList, Timeline,
ReadinessPanel, ApprovalCard,
        |                          # EvidencePopover, CustomerChat,
LearningCurve, LeadTimeBars
        └── lib/format.ts          # ₹ lakh grouping, IST dates
```

Rules that keep the code clean

- Only `memory/client.py` imports the Hindsight SDK, so a version change touches one file.
- Only `llm/groq_client.py` calls Groq, always through the cache.
- The app and the evaluation share `agents/policy.py`, so the evaluation measures the same code the demo shows.
- Prompts live in `agents/prompts/*.md`, not in Python strings.

14. Implementation details

The code below is the skeleton of each key module. SDK calls marked `# VERIFY` follow the public docs and must be checked against the pinned versions on Day 1; everything else is plain Python.

14.1 Memory client (`memory/client.py`)

The only file that touches the Hindsight SDK.

```
from hindsight_client import Hindsight
from munshi.config import settings

class MemoryStore:
    def __init__(self, bank_id: str):
        self.bank_id = bank_id
        self.hs = Hindsight(base_url=settings.hindsight_url,
                            api_key=settings.hindsight_api_key)  # VERIFY

    def retain_event(self, event_id: str, content: str, occurred_on: str,
                     customer_id: str, kind: str) -> None:
        self.hs.retain(
            bank_id=self.bank_id,
            content=content,
            timestamp=occurred_on,           # VERIFY: event time, not ingest
time
            document_id=event_id,            # same id = upsert, safe to
retry
            tags=[f"customer:{customer_id}", f"kind:{kind}"],  # VERIFY
            context="Munshi accounts-receivable event",
        )

    def recall(self, query: str, customer_id: str, fact_types=None):
        return self.hs.recall(bank_id=self.bank_id, query=query,
                              tags=[f"customer:{customer_id}"],   # VERIFY
                              fact_type=fact_types)              # VERIFY

    def reflect(self, query: str, customer_id: str):
        r = self.hs.reflect(bank_id=self.bank_id, query=query,
                            tags=[f"customer:{customer_id}"],    # VERIFY
                            include_facts=True)
        return r.text, r.based_on                                # VERIFY
field names

    def playbook(self, customer_id: str) -> str | None:
        ...  # fetch the customer's mental model by name; VERIFY API
```

14.2 Bank setup (`memory/bank_setup.py`)

```
DIRECTIVES = [
    "Never threaten legal action or use coercive language; escalation needs
owner approval.",
    "Never recommend contacting a customer outside 09:00-19:00 IST.",
    "Never reveal one customer's information to another.",
    "Never state that an invoice is valid or a payment received; the ledger
decides.",
    "Cite the evidence behind every recommendation; if there is none, say
so.",
    "Mention MSME payment-term rules only if the business is marked MSME-
registered.",
]

def ensure_bank(store, business):
    store.hs.create_bank(bank_id=store.bank_id)                    # VERIFY
idempotent
    store.hs.update_bank_config(store.bank_id,
        retain_mission=RETAIN_MISSION,
        observations_mission=OBSERVATIONS_MISSION,
        reflect_mission=REFLECT_MISSION.format(business=business.name),
        disposition_skepticism=4, disposition_literalism=3,
        disposition_empathy=3)                                     # VERIFY
    for text in DIRECTIVES:
        ...  # create each directive; VERIFY API name

def ensure_playbook(store, customer):
    store.hs.create_mental_model(bank_id=store.bank_id,
        name=f"Playbook: {customer.name}",
        source_query=PLAYBOOK_QUERY.format(customer=customer.name),
        tags=[f"customer:{customer.id}"])                           # VERIFY
refresh option
```

14.3 Retain templates (`memory/events.py`)

One function per event kind keeps wording consistent, which helps extraction.

```
def reminder_sent(a, c, inv) -> str:
    return (f"On {a.sent_at:%Y-%m-%d} I (Munshi) sent a {a.channel} reminder
to "
            f"{a.recipient} ({c.contact_role}, {c.name}) about invoice
{inv.number} "
            f"for {rupees(inv.amount_paise)}, due {inv.due_date:%Y-%m-%d}. "
            f"Attached: {', '.join(a.attachments) or 'nothing'}.")

def payment_received(p, c, inv, last_action=None) -> str:
    late = (p.paid_on - inv.due_date).days
    after = (f" {(p.paid_on - last_action.sent_at.date()).days} days after my
reminder to "
             f"{last_action.recipient}" if last_action else "")
    return (f"On {p.paid_on:%Y-%m-%d} {c.name} paid {rupees(p.amount_paise)}
against "
            f"invoice {inv.number}, {late} days {'late' if late > 0 else
'early or on time'}{after}.")

def customer_reply(m, c) -> str:
    return (f"On {m.received_at:%Y-%m-%d} {m.sender_name} ({c.name}) replied
on "
            f"{m.channel}: \"{m.body[:400]}\"")
```

14.4 Policy interface (`agents/policy.py`)

The app and all three evaluation versions implement the same interface, so the evaluation measures the demo's code.

```
from dataclasses import dataclass
from typing import Protocol

@dataclass
class FollowUpPlan:
    send_on: date
    recipient: str
    channel: str
    attachments: list[str]
    tone: str               # "gentle" | "firm" | "thank_you"
    reason: str
    evidence_ids: list[str]

@dataclass
class RiskAssessment:
    level: str              # NORMAL | WATCH | HIGH
    signals: list[str]
```

```
    suggestion: str | None
    evidence_ids: list[str]

class Policy(Protocol):
    def readiness_memory(self, customer, invoice) -> dict: ...
    def plan_followup(self, customer, invoice, today) -> FollowUpPlan | None:
...
    def assess_risk(self, customer, today) -> RiskAssessment: ...
    def observe(self, event) -> None: ...   # A: no-op, B: update columns, C:
retain
```

14.5 Collector, Hindsight version (`agents/collector.py`)

```
class HindsightPolicy:
    def __init__(self, store, llm):
        self.store, self.llm = store, llm

    def plan_followup(self, customer, invoice, today):
        if not needs_attention(invoice, today):         # rule: near due or
overdue
            return None
        if reminders_this_week(invoice) >= 2:           # rule: frequency cap
            return None
        text, based_on = self.store.reflect(
            queries.FOLLOWUP.format(invoice=invoice.number,
customer=customer.name,
                                    days=(today - invoice.due_date).days,
                                    today=today.isoformat()),
            customer.id)
        plan = parse_followup_json(text)                 # validated by
pydantic
        plan.send_on = clamp_to_business_hours(plan.send_on)
        plan.evidence_ids = [m.id for m in based_on]
        return plan
```

The `FOLLOWUP` prompt asks for JSON with `send_on`, `recipient`, `channel`, `attachments`, `tone` and `reason`, and tells the model to choose the contact and timing from evidence. If parsing fails twice, the Collector falls back to a gentle reminder to the contact on file, marked "no memory used".

14.6 Risk agent (`agents/risk.py`)

```
def assess_risk(self, customer, today):
    signals = []
    if missed_payment_runs(customer, today) >= 2:            # from the
ledger
        signals.append("missed two payment runs")
    if broken_promises(customer, days=30) >= 1:
        signals.append("broke a promise to pay")
    obs =
self.store.recall(queries.PAYMENT_BEHAVIOUR.format(customer=customer.name),
                          customer.id, fact_types=["observation"])
    if any(o.trend == "weakening" for o in obs):            # VERIFY trend
field
        signals.append("payment pattern weakening")
    text, based_on = self.store.reflect(queries.RISK_CHANGE.format(...),
customer.id)
    judged = parse_risk_json(text)                           # distress
cues, contact change
    signals += judged.signals
    level = "HIGH" if len(signals) >= 2 else "WATCH" if signals else "NORMAL"
    return RiskAssessment(level, signals, judged.suggestion,
                          [m.id for m in based_on])
```

Ledger signals are computed in code; memory adds the signals only conversations can show (distress language, a contact change, a weakening pattern). The level rule is fixed, so it can be tested.

14.7 Daily loop (`services/daily_loop.py`)

```
def run_day(today, policy, db):
    for inv in open_invoices(db):
        plan = policy.plan_followup(inv.customer, inv, today)
        if plan:
            create_draft(db, inv, plan)                 # status DRAFTED,
posted to Band
    for cust in customers_with_activity(db, since=today - timedelta(days=1)):
        assessment = policy.assess_risk(cust, today)
        if assessment.level != cust.risk_level:
            raise_alert(db, cust, assessment)           # posted to Band by
the bridge
            cust.risk_level = assessment.level
```

In the demo and the evaluation, `/api/sim/tick` calls `run_day` once per simulated day, and the simulator auto-approves drafts only in evaluation runs.

14.8 Groq client with cache (`llm/groq_client.py`)

```
from groq import Groq

class LLM:
    def __init__(self, model: str, cache):
        self.client, self.model, self.cache = Groq(), model, cache

    def json(self, system: str, user: str) -> dict:
        key = hash_prompt(self.model, system, user)
        if (hit := self.cache.get(key)) is not None:
            return hit
        resp = self.client.chat.completions.create(
            model=self.model,                       # e.g. "openai/gpt-oss-
120b"; VERIFY id
            messages=[{"role": "system", "content": system},
                      {"role": "user", "content": user}],
            response_format={"type": "json_object"},
            temperature=0)
        out = json.loads(resp.choices[0].message.content)
        self.cache.set(key, out)
        return out
```

14.9 Band agents (`band/run_agents.py`)

The Band SDK connects each agent through a framework adapter. Use one LLM adapter per agent and give it two tools that call the backend; drafts and alerts are posted by the bridge, not generated by the agent.

```
import asyncio
from thenvoi import Agent                          # package: band-sdk; VERIFY
import path
from thenvoi.adapters import PydanticAIAdapter  # VERIFY adapter and Groq
model string

SYSTEM = ("You are Munshi's Collector on the AR desk. When the owner
approves, edits or "
          "rejects a draft (e.g. 'approve A-231'), call decide_draft. For
questions, call "
          "explain and reply with its answer. Never invent figures.")

async def main():
    adapter = PydanticAIAdapter(model="groq:openai/gpt-oss-120b",
                                system_prompt=SYSTEM,
                                tools=[decide_draft, explain])   # VERIFY
tool registration
    collector = Agent.create(adapter=adapter,
                             agent_id=settings.band_collector_id,
                             api_key=settings.band_collector_key)  # VERIFY
    await collector.run()

asyncio.run(main())
```

`decide_draft` first tries a regex on the message (`approve|edit|reject A-\d+`) and only falls back to the LLM's reading when the regex does not match. The Risk agent is started the same way with its own id, key and an `assess` tool.

15. Simulator

The simulator decides every outcome with seeded, fixed rules; an LLM only writes the customers' reply wording. That keeps results reproducible and stops the LLM from deciding whether Munshi succeeded.

Hidden customer traits

Each simulated customer gets traits the agent never sees directly. It can only learn them from replies and outcomes.

Trait	Example values	How it can be learned	What it affects
Payment-run days	5th and 20th; every Friday; 1st only	Replies ("we release on the 5th") and payment dates	When payment can happen

Trait	Example values	How it can be learned	What it affects
Needs a nudge	Yes, 1 to 5 days before a run	Payments after timely reminders vs slips	Whether an invoice makes a run
Current payables contact	Priya, later Rahul	Replies; ignored reminders	Whether a reminder counts
Required documents	PO_REF, SIGNED_CHALLAN, GRN_COPY, EINVOICE_IRN	Hold notices and replies	Whether an invoice is held
Annoyance threshold	More than 3 reminders in 7 days	Curt replies	One-run payment penalty
Distress onset	None, or a day in month 4	Cue phrases, broken promises, part payments	Skipped runs, possible default
Base lateness	Sampled from the calibrated distribution	Payment dates	Noise not caused by the agent

Outcome rules (`sim/world.py`)

1. Payments happen only on the customer's run days.
2. An invoice joins a run when it is due, not held, and (for nudge-needed customers) a reminder reached the **current** contact 1 to 5 days before the run. Otherwise it slips to a later run; after two slips it is paid anyway, plus base lateness.
3. An invoice sent without a required document is held. The customer names the missing item in a reply (probability 0.8, seeded). The hold clears when an approved DOC_SEND action supplies it.
4. A reminder to an outdated contact has no effect. That person replies with the new contact's name (probability 0.7, seeded).
5. More than 3 reminders in 7 days lowers a relationship score; at −3, payment moves one run later.
6. After distress onset, runs are skipped (probability 0.6, seeded), replies carry cue phrases, and promises are made then broken. One distressed customer defaults on its last two invoices.
7. In month 4, one customer changes a requirement (for example, an e-invoice IRN replaces the signed challan) and says so once.

8. The simulator logs which trait caused each outcome, so metrics can check whether the agent learned the real cause.

Randomness comes from one generator per (seed, customer, day), so the same seed always gives the same world.

Customer replies (`sim/personas.py`)

- **Template mode** (used for the evaluation): fixed phrasings per event, with slots for names and dates. No LLM noise, no API cost.
- **LLM mode** (used for the demo): qwen3-32b plays the customer's accounts person, given their traits and the event. It writes short, informal Indian-English or Hinglish replies, reveals at most one hint per message, and never lists its rules. Replies are cached by (seed, customer, event).
- **Live mode** (demo option): the presenter or a judge types the customer's replies in the Simulation console.

Calibration (`sim/calibrate.py`)

Base lateness is drawn from IBM's public sample accounts-receivable dataset of 2,466 invoices from 100 customers (dataset description). The script fits a days-late distribution per customer and samples 12 of them as base profiles. Download the dataset separately and check its licence before committing it.

The 2 to 3 supplier interviews (section 18) set which documents, contact patterns and excuses appear, so the traits reflect real Indian invoicing rather than guesses.

Scenarios

File	Customers	Length	Purpose
`demo.yaml`	4	4 months	The scripted story: Reddy Traders' contact change and distress, one requirement change, one steady customer
`eval.yaml`	12	6 months	Learning in months 1 to 3; in month 4, one contact change, one requirement change, two distress onsets

```
# sim/scenarios/demo.yaml (excerpt)
start: 2026-01-01
months: 4
customers:
  - id: reddy
    name: Reddy Traders
    gstin: 36AAACR1234A1Z5        # illustrative; generate valid test GSTINs
with gstin.py
    credit_days: 30
    run_days: [5, 20]
    needs_nudge: true
    contacts: [{name: Priya, role: Accounts, until: 2026-04-01}, {name:
Rahul, role: Payables}]
    required_docs: [PO_REF, SIGNED_CHALLAN]
    distress_onset: 2026-04-10
    invoices_per_month: 2
    amount_range_rupees: [25000, 80000]
```

16. Evaluation protocol

Three versions run on the same simulated customers and seeds; the claim is only as strong as C's lead over B. If C does not beat B, the results say so and the team changes what memory holds.

The three versions

Version	What it knows	How it decides
A: fixed schedule	Only the contact on file at the start	Reminders on the due date, +7 and +14 days; rule checks only
B: structured memory	Ledger statistics (average days late, usual payment day of month) plus an LLM that extracts each reply into fixed columns: contact, run days, required documents, promise date	Same drafting LLM and rules as C, given those columns
C: Munshi with Hindsight	Everything retained in Hindsight	Reflect, recall, observations and mental models, as in sections 7 and 14

B is deliberately strong. It is the "just use a database" answer a judge will raise, so beating it is the real test.

Metrics (test months 4 to 6 only)

Metric	Definition	Better
Average days late	Mean of (paid date − due date) over paid invoices	Lower
Reminders per paid invoice	Approved reminders ÷ paid invoices	Lower
Wrong-contact reminders	Reminders sent to an outdated contact	Lower
Blockers caught before sending	Invoices that would have been held, flagged before sending ÷ all such invoices	Higher
False blockers	Blocker flags on invoices that needed nothing more	Lower
Requirement-change lag	Invoices sent after a requirement change before the version adapts	Lower
Distress lead time	Days between the first High alert and the first missed run caused by distress	Higher
False High alerts	High alerts on customers with no distress	Lower
Trait recovery	Share of hidden traits correctly stated in the version's customer summary (checked against simulator logs)	Higher
Evidence grounding (C only)	Share of suggestions whose evidence ids point to the event that actually caused the trait	Higher

Procedure

1. Generate the `eval.yaml` world for seeds 1, 2 and 3.
2. For each seed, run A, B and C through all six months. C gets a fresh bank per run (`eval-<run>-C-s<seed>`).
3. Use template replies, so only the agents differ. Drafts are auto-approved in evaluation runs.
4. Before each simulated day's decisions in C, wait until Hindsight reports consolidation is up to date (or a 30-second timeout, logged).
5. Score months 4 to 6 only. Months 1 to 3 are learning time.
6. Write `eval/results/results.json` and the charts; commit them with the git hash of the code that produced them.

Reporting

- One table: each metric for A, B and C as the mean across seeds, with the min–max range.
- One chart: average days late per month for A, B and C, test months shaded.
- One chart: distress lead time per version.
- A short "what we found" paragraph in `docs/evaluation.md`, including any metric where C lost.

Cost and time (estimate, check in the Day 1 to 2 test)

The 12-customer, 6-month world produces about 140 invoices and roughly 600 retained events per seed for C, plus several hundred reflect calls. Each retain also uses LLM calls for extraction. Run the full evaluation at most twice, overnight, one seed at a time, and keep the cache. If Groq's limits are too tight, cut to 8 customers and 2 seeds and say so in the results.

If C does not beat B

Look at which metrics it lost. Usually that means memory is holding structured facts that a column handles well. Move those to SQLite and focus Hindsight on what columns miss: why customers pay, distress language, conflicting evidence and changes over time. Then re-run.

17. Demo script and runbook

The value must land in 60 seconds, so the demo shows one customer changing over time, the learning curve, and one belief weakening.

The 60-second core

Time	Screen	What is said	What judges see
0–8 s	Dashboard	"MSMEs wait 73 days to get paid. Every owner keeps a mental map of who pays when. Munshi learns it."	Overdue total, at-risk customers
8–20 s	New invoice	"Before this invoice goes out, Munshi remembers Reddy Traders held the last two until a signed challan arrived."	Learned blocker, "Why?" evidence

Time	Screen	What is said	What judges see
20–32 s	Customer page	"Month 1, a generic reminder. Month 4, the right person, two days before their payment run."	Two drafts side by side, payment on the 5th
32–45 s	Evaluation	"Across 12 customers: no memory, database memory, and Hindsight."	Learning curve, A vs B vs C
45–57 s	Band room	"When the pattern weakens, the Risk agent warns the owner days before the default."	Belief turns to weakening, Band alert
57–60 s	—	"Turn off Hindsight and it's a reminder bot again."	—

Fill the evaluation line with the real numbers from the final run. If C did not win a metric, don't show that metric as a win.

Full video (2 to 3 minutes)

1. The 60-second core.
2. How memory works: the playbook card, beliefs with trends, the evidence trail (30 s).
3. Under the hood: architecture diagram, the bank config, what we verified about Hindsight (30 s).
4. The evaluation: method, results table, honest limits (30 s).
5. Close: who this is for, what's next (10 s).

Live demo runbook

1. **Night before:** run `scripts/prewarm_demo_bank.py` into a fresh bank (`munshi-demo-final`), so live consolidation never blocks the demo. Export results and screenshots.
2. **One hour before:** check Hindsight Cloud, Groq and Band are reachable; start the backend, frontend and Band agents; open the five tabs in order.
3. **Live part:** reset to the prewarmed state, show the 60-second core, then offer the judge interaction.
4. **Judge interaction (optional):** the judge types as Reddy Traders' accountant (for example, "we only pay after GRN approval"). Fast-forward 30 days. The next draft and the readiness check reflect it.
5. **If anything fails:** switch to the recorded video at the same timestamp. Never debug in front of judges.

Backup plan by failure

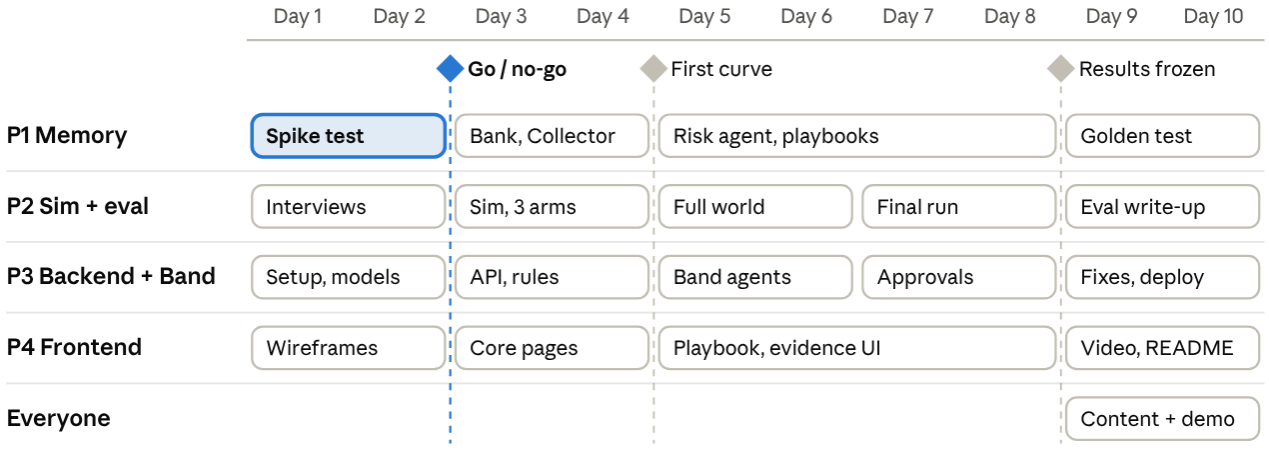
Fails	Switch to
Hindsight Cloud	Local Docker Hindsight with a copy of the prewarmed bank
Groq	Cached responses (the demo path is fully cached after one rehearsal)
Band	Web Approvals inbox and Risk page, plus the Band screen recording
Laptop or network	The full video on a second laptop and a phone hotspot

18. Build plan, team split and gates

The build runs about 10 days with three gates. The Day 2 test decides whether the memory design works before anyone builds UI around it.

A go/no-go gate on Day 2 decides the build before any UI work

10-day plan in relative days; set real dates once the deadline is confirmed



Build plan · 4 owners, 10 relative days, 3 gates

Days are relative because the deadline is not yet confirmed. Compress the middle phases if it is shorter, never the Day 1 to 2 test or the rehearsals.

Owners

Person	Owns	Folders
P1, memory lead	Hindsight bank, retain templates, queries, Collector and Risk logic, golden scenario	`memory/`, `agents/`
P2, simulator and evaluation	Supplier interviews, hidden traits, outcome rules, three versions, metrics and charts	`sim/`, `eval/`
P3, backend and Band	Data model, API, rules R1 to R9, Band agents, bridge and approvals	`db/`, `api/`, `checks/`, `band/`, `services/`
P4, frontend and story	All pages and components, demo script, video, README	`frontend/`, `docs/`

Gates

1. **End of Day 2, go/no-go.** One simulated customer over 6 months: Hindsight forms the right beliefs, reflect changes the next plan, and a belief weakens after the contact change. If not, apply the fallbacks in section 7 before going further.
2. **End of Day 4, first curve.** A, B and C run end to end on a small world (6 customers, 1 seed). C must show a visible lead over B on at least two metrics, or the team changes what memory holds (section 16).
3. **End of Day 8, results frozen.** Final evaluation committed with its git hash. After this, only bug fixes, polish, the video and content.

Supplier interviews (Day 1 to 2, P2)

Call 2 to 3 small business owners or accountants. Ask one opening question: "Tell me about the last invoice that got stuck. Why, and how did you find out?" Then ask who releases payments, which documents buyers insist on, and how they notice a customer in trouble. Use the answers to set the simulator's traits and quote them in the articles.

19. Setup and running

A new team member should go from clone to a running demo in under 15 minutes with four commands.

Prerequisites

Python 3.11+, uv, Node 20+, Docker, a Groq API key, Hindsight Cloud access or local Docker, and a Band account with two External Agents created.

Environment variables (`.env.example`)

Variable	Example	Used by
GROQ_API_KEY	gsk_…	Agents, simulator, Hindsight (if self-hosted)
LLM_AGENT_MODEL	openai/gpt-oss-120b	Collector, Risk, drafting (check the exact Groq model id)
LLM_SIM_MODEL	qwen/qwen3-32b	Customer personas (check the exact Groq model id)
HINDSIGHT_URL	http://localhost:8888	Memory client
HINDSIGHT_API_KEY	(Cloud only)	Memory client
HINDSIGHT_BANK_PREFIX	munshi	Bank naming
BAND_COLLECTOR_AGENT_ID / BAND_COLLECTOR_API_KEY	from Band	Collector agent
BAND_RISK_AGENT_ID / BAND_RISK_API_KEY	from Band	Risk agent
BAND_ROOM_ID	from Band	Bridge
DATABASE_URL	sqlite:///./munshi.db	Backend
SIM_MODE	true	Simulated clock and simulator endpoints
LLM_CACHE_DIR	.cache/llm	LLM response cache
TZ	Asia/Kolkata	Business hours and date display

Run Hindsight locally

```
docker run --rm -it --pull always -p 8888:8888 -p 9999:9999 \
  -e HINDSIGHT_API_LLM_PROVIDER=groq \
  -e HINDSIGHT_API_LLM_API_KEY=$GROQ_API_KEY \
  -e HINDSIGHT_API_LLM_MODEL=openai/gpt-oss-120b \
  -v $HOME/.hindsight-docker:/home/hindsight/.pg0 \
  ghcr.io/vectorize-io/hindsight:latest
# API on :8888, web UI on :9999. Check env var names against the Hindsight
README.
```

Hindsight's own web UI on port 9999 shows the bank's facts, observations and mental models. Use it in the "how memory works" part of the video.

Run Munshi

```
cp .env.example .env                        # fill in keys
make dev        # backend on :8000 (uvicorn) and frontend on :5173 (vite)
make seed       # demo business, customers, fresh bank, mental models
make agents     # Collector and Risk agents connect to the Band room
```

Run the tests and the evaluation

```
make test       # unit tests; integration tests run if HINDSIGHT_URL is
reachable
make eval       # = uv run python -m eval.run_eval --arms A,B,C --seeds 1,2,3
--replies template
make spike      # = uv run python scripts/spike_one_customer.py (the Day 1-2
test)
```

docker-compose.yml (outline)

```
services:
  hindsight:
    image: ghcr.io/vectorize-io/hindsight:latest   # pin a version tag on Day
1
    ports: ["8888:8888", "9999:9999"]
    env_file: .env
    volumes: ["hindsight-data:/home/hindsight/.pg0"]
  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [hindsight]
  frontend:
    build: ./frontend
    ports: ["5173:5173"]
    depends_on: [backend]
volumes:
  hindsight-data:
```

20. Testing strategy

Fixed rules and the simulator get full unit tests; memory gets integration tests and golden scenarios, because its output is not exactly repeatable.

Layer	Tests	Pass condition
GSTIN and rules	`test_gstin.py`, `test_rules.py`: valid and invalid GSTINs, each rule R1 to R9	All cases pass; runs in CI on every push
Readiness combiner	`test_readiness.py`: rule blocker, learned blocker, conflict, uncertain, ready, using fake memory findings	Each input gives the expected result
Simulator rules	`test_world_rules.py`: same seed twice gives identical outcomes; each rule triggers as described	Deterministic, all rules covered
Retain templates	`test_events.py`: absolute dates, first-person actions, no phone or bank numbers	All templates pass the checks
Reflect parsing	`test_parsing.py`: valid JSON, malformed JSON, missing fields	Malformed output falls back safely

Layer	Tests	Pass condition
Memory integration	`test_memory_integration.py` against a running Hindsight: retain an event, recall finds it; tagged recall excludes other customers	Passes against the pinned version
Golden scenario	Run `demo.yaml`; check that Reddy Traders' playbook mentions Rahul, the 5th and 20th, and the signed challan by month 4	All four facts present
End to end	Playwright smoke test: create invoice, see blocker, approve a draft	Passes before each rehearsal

Run the golden scenario after any prompt or Hindsight version change. If it fails, fix that before touching anything else.

21. Risks, limitations and safeguards

The biggest risks are technical (Hindsight behaviour under simulated time, API limits) and presentational (overclaiming); each has a fallback.

Risks

Risk	Likelihood	Impact	Mitigation
Hindsight ignores supplied timestamps, so trends follow ingest time	Medium	High	Found in the Day 1–2 test; fallback is date-aware reflect prompts and ledger-based trend signals
Observations or mental models lag during the demo	High	Medium	Prewarmed bank; "memory updating" shown honestly; wait for consolidation in evaluation
Groq or Hindsight rate limits during evaluation	Medium	Medium	Cache, overnight runs, smaller world if needed, reported openly
C does not beat B	Medium	High	Section 16 fallback: refocus memory on what columns miss, re-run

Risk	Likelihood	Impact	Mitigation
Band SDK differs from the docs	Medium	Low	Web inbox does the same job; Band integration is a thin layer
LLM drafts contain wrong facts	Low	High	Amounts and dates are filled from the ledger by code, not written by the LLM
Scope creep	High	High	Out-of-scope list below; any new feature needs the whole team's yes

Honest limitations to state in the demo

- The customers are simulated. Their patterns are based on a public dataset and a few interviews, not real customer data.
- In the real world you can't prove a reminder caused a payment without an A/B test.
- Hindsight's extraction is not fully deterministic, so results vary slightly between runs. Results are reported across three seeds.
- The evaluation measures behaviour in one simulated world, not money recovered by a real business.

Safeguards

- **Business-to-business only.** Munshi is not for consumer debt collection.
- **Human approval on every outgoing message.** No automatic sending; real use goes through the owner's own WhatsApp.
- **Directives** block threats, out-of-hours contact and cross-customer disclosure (section 7).
- **No credit decisions by the agent.** The Risk agent suggests; the owner decides.
- **Data minimisation.** Phone and bank numbers stay in SQLite, never in memory. One bank per business. Deleting a customer deletes their memories by document id.
- **Legal wording.** Any mention of MSME payment rules is a fixed, CA-reviewed template, shown only when the owner has marked the business as registered.

Out of scope

Real WhatsApp API, Tally integration, OCR of scanned invoices, payment processing, buyer portal submission, multi-user login, mobile layout, Dispute agent (stretch only).

22. Submission checklist and open items

Five things must be confirmed with the organisers this week; the rest is a checklist to tick off before submitting.

Confirm with the organisers

- [ ] Submission deadline and live-demo slot
- [ ] What Band usage is expected (product integration, development tool, or both)
- [ ] The official content guide: article length, platform, social post and video format per member
- [ ] Hindsight Cloud credit eligibility (MEMHACK99) and any usage limits
- [ ] Whether simulated data is acceptable, or real data is preferred

Also check

- [ ] The name "Munshi" is not taken by a similar product
- [ ] CredFloat, named in an earlier review, was not found in our search; check its features directly
- [ ] The IBM sample receivables dataset's licence before committing it to the repo

Repository

- [ ] README: problem, 30-second GIF, architecture diagram, "How Munshi uses Hindsight" section, how to run, evaluation results
- [ ] `docs/hindsight.md`: bank config, retain templates, queries, what the Day 1–2 test found
- [ ] `docs/evaluation.md` with the final results table and charts
- [ ] `.env.example` with no secrets; no keys anywhere in history
- [ ] Tests pass; `make eval` reproduces the committed results
- [ ] Licence file

Demo and video

- [ ] 60-second core rehearsed at least twice, under 60 seconds
- [ ] Full 2 to 3 minute video recorded and uploaded
- [ ] Prewarmed demo bank and a local copy
- [ ] Backup video on a second laptop

Content from every member

Each member writes from their own module, so the four articles don't repeat each other.

Member	Article angle
P1, memory lead	How we designed memory that unlearns: missions, directives, observations and trends
P2, simulator and evaluation	Proving an agent learns: the three-version test and what surprised us
P3, backend and Band	Two agents and a human in one Band room: approvals and handoffs
P4, frontend and story	Making memory visible: designing the playbook card and the "Why this?" panel

- [ ] Four articles published
- [ ] Four social posts with the demo GIF
- [ ] Four short videos

23. References

Pages used for facts in this document, as of 6 November 2026.

Market and law

- Recordent Indian SME Receivables Report 2026, reported by KNN
- Whalesbook: India's big firms use MSME payment delays for free cash
- TaxGuru: MSME delayed payment reporting and Section 43B(h)

Hindsight

- Hindsight docs: memory types and overview
- Hindsight docs: reflect, directives and disposition
- Hindsight blog: observations and mental models
- Hindsight blog: staleness-aware memory
- Hindsight docs: observation freshness trend
- Hindsight on GitHub

Band

- Band for Developers launch announcement
- Band Python SDK on PyPI

- Band docs: connect a remote agent

Competitors and data

- Upflow cash collection features
- HighRadius collections review
- IBM sample accounts-receivable dataset, as described in a project write-up